

ROS2 day3 hw1 레포트

로봇 예비단원 주호진

1. 과제

QT 신호등 + 교차로 시뮬레이터

yaml에서 신호등의 점등 시간, 차량의 속도 등 관리

노드

1: traffic light node - 신호등 상태 관리 RED → GREEN → YELLOW → RED

2: vehicle node

- 차량 생성 및 이동

GREEN → 이동

YELLOW → 감속/정지

RED → 정지

Qt에서 로직 처리 X, 시뮬레이션 시각화만 담당

- Topic을 받아서 현재 신호와 차량 위치를 Qt 화면에 실시간으로 표시

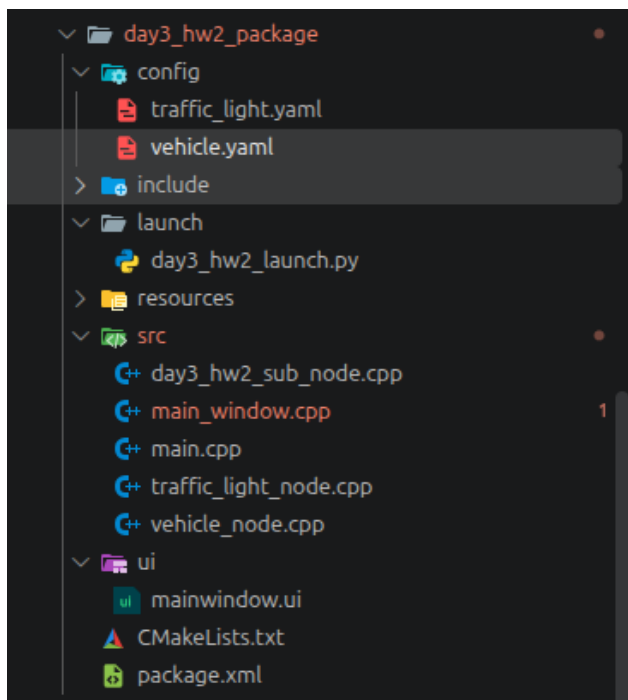
-> 작동 영상 첨부 (파라미터 변경 전·후 과정을 한 영상에 포함)

2. 코드

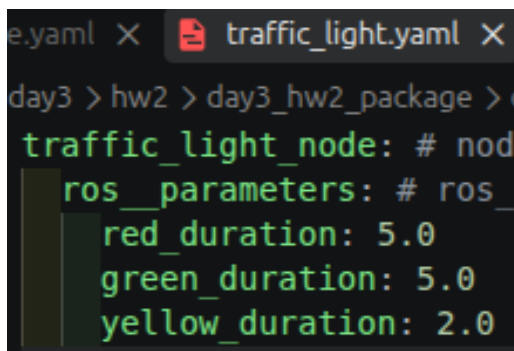
- 폴더 구조

폴더 구조는 다음과 같이 되어있다. 아래 코드 설명은 주요 부분을 위주로 넣었다.

(간단하거나 앞선 과제에서 메인으로 상세하게 다뤘던 부분은 뺐다)



- Traffic Light Node (점등 시간 node)



신호별 점등 시간을 가지고 있는 yaml 파일을 만들었다.

```

class TrafficLightNode : public rclcpp::Node
{
public:
    TrafficLightNode();

private:
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr state_publisher_;
    rclcpp::TimerBase::SharedPtr timer_;

    std::string current_state_;
    double red_duration_;
    double green_duration_;
    double yellow_duration_;
    double elapsed_time_{0.0};

    void timer_callback();
};

```

시간을 가지고 있는 yaml 값을 읽어서 해당 시간만큼 red, yellow, green을 퍼블리시 해주는 노드이다.

```

TrafficLightNode::TrafficLightNode() : Node("traffic_light_node")
{
    state_publisher_ = create_publisher<std_msgs::msg::String>("topic_traffic", 10);

    declare_parameter<double>("red_duration", 5.0);
    declare_parameter<double>("green_duration", 5.0);
    declare_parameter<double>("yellow_duration", 2.0);

    red_duration_ = get_parameter("red_duration").as_double();
    green_duration_ = get_parameter("green_duration").as_double();
    yellow_duration_ = get_parameter("yellow_duration").as_double();

    current_state_ = "GREEN";
    timer_ = create_wall_timer(std::chrono::milliseconds(16), std::bind(&TrafficLightNode::timer_callback, this));
}

```

생성자는 파라미터 기본값을 설정해주고 yaml 파일 값을 읽어온다. 맨 처음 신호등 색은 녹색으로 설정해주었다. 좌표만을 전달해주기 때문에 timer 갱신이 너무 느리면 끊겨서 보일 것 같아서 60Hz에 맞추어 16ms로 설정했다. (1000ms / 60 = 16ms)

```

void TrafficLightNode::timer_callback()
{

```

```

    double state_duration = 0.0;

```

```

if (current_state_ == "RED")
    state_duration = red_duration_;
else if (current_state_ == "GREEN")
    state_duration = green_duration_;
else
    state_duration = yellow_duration_;

```

state를 읽고 yaml파일에서 읽어온 점등 시간을 state_duration에 설정한다.

```

const double timer_period = 0.1;
elapsed_time_ += timer_period;

```

```

if (elapsed_time_ >= state_duration) {
    // red -> green -> yellow -> red
    elapsed_time_ = 0.0;
    if (current_state_ == "RED")
        current_state_ = "GREEN";
    else if (current_state_ == "GREEN")
        current_state_ = "YELLOW";
    else
        current_state_ = "RED";
}

```

타이머가 해당 점등 시간만큼 지나게 되면 state를 다음 신호로 바꾼다. // r -> g -> y -> 다시 r

```

std_msgs::msg::String message;
message.data = current_state_;
state_publisher_ -> publish(message);

```

해당하는 신호등의 색을 publish한다.

- vehicle Node (좌표 계산)

```

vehicle.yaml x traffic_light.yaml x mainv
ROS2 > day3 > hw2 > day3_hw2_package > config >
1 vehicle_node: # node name
2   ros_parameters: # ros_paramet
3     start_x: 0.0
4     start_y: 5.0
5     direction: "x"
6     stop_line_position: 5.0 # 일단
7     road_length: 7.0
8     speed: 1.0
9     acceleration: 0.5
10    deceleration: 0.5
11    emergency_deceleration: 2.0
12    max_speed: 2.0
13

```

파라미터를 꽤 많이 설정했는데 처음에는 커스텀 메시지로 묶어서 여러대를 띄우고 x축을 길로 움직이는 vehicle과 y축을 길로 움직이는 vehicle을 각각 놔서 교차로를 구현하려고 했다. 시간상 그렇게 하지는 못했다.

```

class VehicleNode : public rclcpp::Node
{
public:
    VehicleNode();
}

```

노드는 일단 해당 yaml 파일에서 받아온 값을 바탕으로 속도와 가속도 등을 계산한다. 속도를 계산하기 위해서는 주행체가 어떤 신호를 인식하고 있는지 알아야 하기 때문에 위에서 설정한 신호 등 node의 publish를 받아온다(sub).

```

void traffic_callback(const std_msgs::msg::String::SharedPtr message);
rclcpp::Subscription<std_msgs::msg::String>::SharedPtr traffic_subscription_;
std::string current_light_{"GREEN"};

```

Vehicle node 생성자는 일단 신호등 상태를 받아오는 pub과 파라미터 기본값 설정 및 파라미터 값을 받아오기, direction값에 따라 x축으로 움직이는 객체인지 y축으로 움직이는 객체인지 판단하도록 해놓았고, 해당 노드에서 좌표값을 publish하게 만들었다.

```

VehicleNode::VehicleNode() : Node("vehicle_node")
{
    position_publisher_ = create_publisher<geometry_msgs::msg::Pose2D>("topic_vehicle", 10);
    traffic_subscription_ = create_subscription<std_msgs::msg::String>(
        "topic_traffic", 10, std::bind(&VehicleNode::traffic_callback, this, std::placeholders::_1));
    declare_parameter<double>("start_x", 0.0);
    declare_parameter<double>("start_y", 10.0);
    declare_parameter<double>("speed", 1.0);
    declare_parameter<std::string>("direction", "x");
    declare_parameter<double>("stop_line_position", 0.0);
    declare_parameter<double>("road_length", 20.0);
    declare_parameter<double>("acceleration", 0.5);
    declare_parameter<double>("deceleration", 0.5);
    declare_parameter<double>("emergency_deceleration", 2.0);
    declare_parameter<double>("max_speed", 2.0);

    start_x_ = get_parameter("start_x").as_double();
    start_y_ = get_parameter("start_y").as_double();
    direction_ = get_parameter("direction").as_string();
    stop_line_position_ = get_parameter("stop_line_position").as_double();
    road_length_ = get_parameter("road_length").as_double();
    speed_ = get_parameter("speed").as_double();
    acceleration_ = get_parameter("acceleration").as_double();
    deceleration_ = get_parameter("deceleration").as_double();
    emergency_deceleration_ = get_parameter("emergency_deceleration").as_double();
    max_speed_ = get_parameter("max_speed").as_double();

    x_ = get_parameter("start_x").as_double();
    y_ = get_parameter("start_y").as_double();

    current_velocity() = get_parameter("speed").as_double();

    timer_ = create_wall_timer(std::chrono::milliseconds(16), std::bind(&VehicleNode::timer_callback, this));
}

double &VehicleNode::current_velocity()
{
    if (direction_ == "x")
        return vx_;
    return vy_;
}

double &VehicleNode::current_position()
{
    if (direction_ == "x")
        return x_;
    return y_;
}

```

아까 위에서 언급하긴 했지만 direction값에 따라 어느 축을 움직이는지 판별하도록 설정해놨다.

```

if (current_light_ == "GREEN")
{
    velocity += acceleration_ * dt; //  $v = v_0 + at$ 
    if (velocity > max_speed_) // 최대 속도인 경우 속도 고정
        velocity = max_speed_;
}

```

Green인 경우 직진만 하면 되기 때문에 max_speed까지 가속한다.

```

}
else if (current_light_ == "YELLOW")
{
    const double distance_to_stop_line = stop_line_position_ - position; // 정지선까지 거리
    const double stopping_distance = velocity * velocity / (2.0 * deceleration_); // 제동 거리( $v^2 = 2as$  이항)

    // 정지 가능한 경우에 정지, 아니면 그냥 통과
    if (distance_to_stop_line > 0.0 && distance_to_stop_line <= stopping_distance)
    {
        velocity -= deceleration_ * dt;
        if (velocity < 0.0)
            velocity = 0.0;
    }
}
}

```

노란불의 경우 정지선과 현재 위치의 좌표를 통해 정지선과 얼마나 떨어져 있는지를 먼저 체크한다. 너무 멀리 있는 경우에는 노란불이라고 해도 감속을 하지 않아도 되기 때문이다.

Stopping distance는 평범하게 감속했을 때 딱 정지선에 멈출 수 있는 최적의 거리이다. Yaml 파일에서 가속도(감속도)가 정해져 있기 때문에 정확한 값이 나온다. 일단 해당 거리 안쪽으로 들어 오기 시작하면 감속을 진행한다.

```

}
else if (current_light_ == "RED")
{
    // 빨간 불인데 아직 통과 못한 경우
    if (position < stop_line_position_)
    {
        const double distance_to_stop_line = stop_line_position_ - position; // 정지선까지 거리
        const double normal_stopping_distance = velocity * velocity / (2.0 * deceleration_); // 일반 제동 거리
        const double emergency_stopping_distance =
            velocity * velocity / (2.0 * emergency_deceleration_); // 급제동 거리

        if (distance_to_stop_line < normal_stopping_distance &&
            distance_to_stop_line >= emergency_stopping_distance)
        {
            velocity -= emergency_deceleration_ * dt;
        }
    }
}
}

```

노란불을 Stopping distance 보다 더 뒤에서 감지했을 때는 부드럽게 제동이 가능하다. 그러나 Stopping distance보다 짧은 거리에서 노란불로 바뀐 경우에는 정지선까지 정해진 감속으로 멈출 수 없게 되고, 빨간불을 만나게 되는데 이때, 급제동(가중치 감속)을 할 건지, 아니면 빠르게 통과 할 것인지 판단하게 된다.

1차원 상의 운동이기 때문에 아주 간단한 일반 역학에서 나오는 거리, 속도, 가속도에 관한 식들을 이항해서 적용해주면 계산이 간편하다. 사실상 아래의 공식만 쓰면 끝이다.

$$v = v_0 + at$$

$$s = v_0 t + \frac{1}{2} at^2$$

$$2as = v^2 - v_0^2$$

```
const double next_position = position + velocity * dt; // 다음 위치
// (v^2 = 2as 이항) 급제동 시 정지선 앞에 멈출 수 있는지
const bool can_stop_at_red_line =
    position < stop_line_position_ && velocity > 0.0 &&
    velocity * velocity / (2.0 * emergency_deceleration_) <= stop_line_position_ - position;

if (current_light_ == "RED" && can_stop_at_red_line && next_position >= stop_line_position_)
{
    position = stop_line_position_;
    velocity = 0.0;
}
else
{
    position = next_position;
}
```

만약 빨간 불에서 stop line에 정지 가능한 경우로 도달했을 때 앞으로 가지 않도록 한다. 그게 아니라면 통과하도록 한다.

```
geometry_msgs::msg::Pose2D message;
message.x = x_;
message.y = y_;
message.theta = 0.0;
position_publisher_ -> publish(message);
```

해당 노드에서 연산을 전부 진행하고 qt에는 x, y의 좌표만 넘겨주게 된다.

원래는 stop line 위치 값도 넘겨주어야 맵을 그릴 때 stop line이 어디인지 인식하고 자동으로 교차점을 그릴 수 있는데, 시간 상 수정하지 못했다.

- sub Node (qt 받아오기)

```

class Day3Hw2SubNode : public rclcpp::Node
{
public:
    Day3Hw2SubNode();

    std::string traffic_state()
    {
        return traffic_state_;
    }

    geometry_msgs::msg::Pose2D vehicle_position()
    {
        return cur_coordinate_;
    }
}

```

```

class MainWindow : public QMainWindow
{
public:
    MainWindow();

```

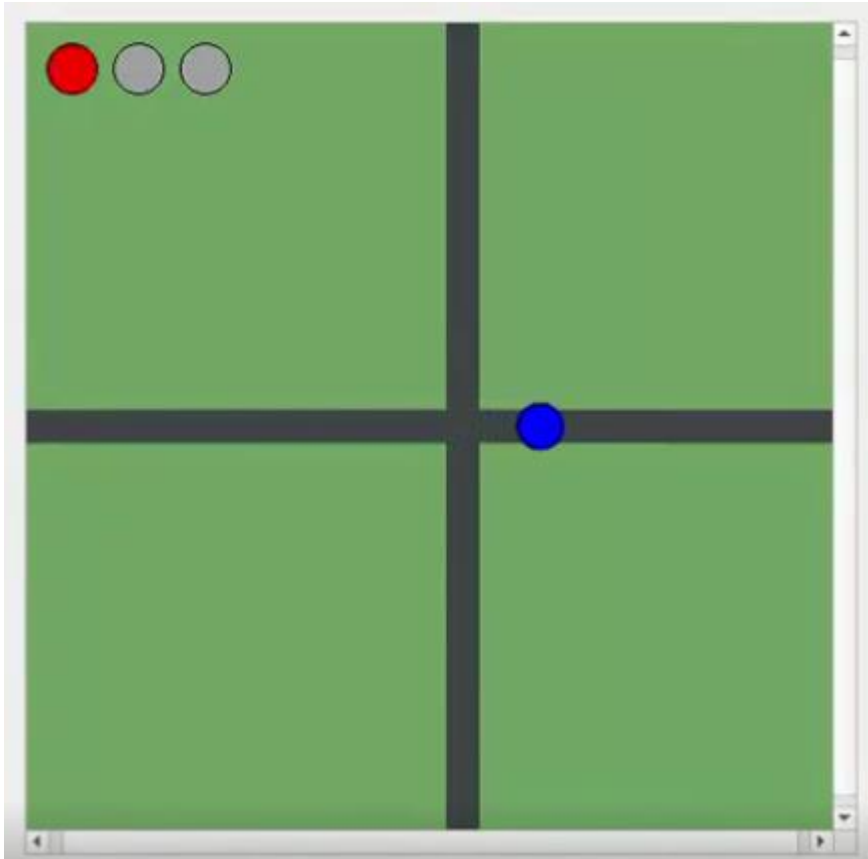
```

    std::shared_ptr<Day3Hw2SubNode> sub_node_;
}

```

Qt에는 위에 계산한 좌표값(x, y)과, 신호등의 상태 총 3개의 값을 받는다.

- mainWindow(ui)



이 부분을 구현할 시간이 부족해서 AI를 사용해서 그린 뒤 수치 값만 수정했다. QGraphicsView을 사용해야 하는데 해당 사용법을 익히면 과제를 제출하지 못할 것 같아서 AI를 사용했다.

```
scene_ ->addRect(0.0, 240, 500.0, 20.0, QPen(Qt::NoPen), QBrush(QColor("#3f4448")));
scene_ ->addRect(260.0, 0.0, 20.0, 500.0, QPen(Qt::NoPen), QBrush(QColor("#3f4448")));

red_light_item_ = scene_ ->addEllipse(20.0, 20.0, 30.0, 30.0, QPen(Qt::black), QBrush(Qt::gray));
yellow_light_item_ = scene_ ->addEllipse(60.0, 20.0, 30.0, 30.0, QPen(Qt::black), QBrush(Qt::gray));
green_light_item_ = scene_ ->addEllipse(100.0, 20.0, 30.0, 30.0, QPen(Qt::black), QBrush(Qt::gray));
vehicle_item_ = scene_ ->addEllipse(0.0, 0.0, 28.0, 28.0, QPen(Qt::black), QBrush(Qt::blue));
```

Sub node를 통해 받아온 값을 업데이트 해준다. 신호등 색 state에 따라 왼쪽 위의 색상이 변하도록 되어있고(AI가 짜주었다), 위 사진에는 나와있지 않지만 map 하단에 label을 추가하여 x, y 좌표값을 출력하도록 했다.

```
void MainWindow::update_visualization()
{
    const std::string state = sub_node->traffic_state();
    red_light_item->setBrush(QBrush(state == "RED" ? Qt::red : Qt::gray));
    yellow_light_item->setBrush(QBrush(state == "YELLOW" ? Qt::yellow : Qt::gray));
    green_light_item->setBrush(QBrush(state == "GREEN" ? Qt::green : Qt::gray));

    const auto position = sub_node->vehicle_position();
    constexpr double world_scale = 40.0;
    const double scene_x = 50.0 + position.x * world_scale;
    const double scene_y = 450.0 - position.y * world_scale;
    vehicle_item->setPos(scene_x - 14.0, scene_y - 14.0);

    QString str = QString("X: %1, Y: %2").arg(position.x, 0, 'f', 2).arg(position.y,
    ui->label->setText(str);
}
```

3. 작동 영상

<https://drive.google.com/file/d/1D9SMcRulydJw5XRAj8k8PUAm8Yd1eihM/view?usp=sharing>

4. AI 보고서

<https://share.gemini.google/vDbDq7ypvOJx>

yaml에서 신호등의 점등 시간, 차량의 속도 등 관리
노드

1: traffic light node - 신호등 상태 관리

RED → GREEN → YELLOW → RED

2: vehicle node - 차량 생성 및 이동

GREEN → 이동

YELLOW → 감속/정지

RED → 정지

Qt에서 로직 처리 X, 시뮬레이션 시각화만 담당

- Topic을 받아서 현재 신호와 차량 위치를 Qt 화면에 실시간으로
표시

이렇게 나와있는데 yaml에서 뭘 관리해야할까

일단 vehicle은 속도랑 가속도

traffic은 점등시간이랑 불빛 색 넣었는데 더 추가로
관리할만한거있나

^

차량과 신호등 시뮬레이션의 확장성과 유연성을 높이기 위해 YAML 파일에 추가하면 유용한
파라미터들입니다.

신호등(Traffic Light) 관련 추가 항목

- 정지선 위치 (Stop Line Position): 차량이 멈춰야 하는 도로 상의 기준 좌표.
- 초기 상태 (Initial State): 시뮬레이션 시작 시 최초 점등 색상 (RED / GREEN).
- 신호등 ID / 위치: 다중 교차로 확장을 대비한 신호등 고유 번호와 배치 좌표.

Yaml 파일에서 어떤 옵션을 관리해야할지 질문했다. 처음에 확실히 정한 다음에 node를 구현하
려고 했는데, 하다보니까 파라미터가 점점 추가되는 형식으로 짜게 됐다.

qt에서는 pub만 받아서 띄우는 거면
sub하기 전에 로직을 다 계산해서 보내면 되나



로직을 pub하기 전에 다 수행하는 거니까 사실상 신호등 state랑 차량 좌표만 넘겨주면 되는건가? 속력도 pub 하기 전에 처리하는 게 맞는 것 같은데\

Qt에서 로직 처리 X, 시뮬레이션 시각화만 담당이라는 조건이 있었기 때문에 qt상에서 속력을 받아서 계산을 해도 되는지 질문했다. 속력을 받아서 구현하는 부분은 시각화에 해당한다고 판단하고 그렇게 하려고 했으나, 그냥 timer duration을 짧게 해서 부드럽게 움직이는 것처럼 보이게 하는 방향으로 생각을 바꿨다.

교차로에서 멈추고 가는 걸 구현하려면 여러대가 있어야 할텐데 이러면 커스텀 메세지 형태로 넘겨줘야 하나?

여러 대의 차량을 효율적으로 관리하고 시각화하려면 개별 데이터를 하나로 묶은 **커스텀 메세지(Custom Message)** 구조를 사용하는 것이 가장 적합합니다.

교차로 구현을 할 때 여러대를 띄우는 방법으로 커스텀 메세지 외 다른 방법도 있다고 AI가 제시해주었는데, 시간 관계상 여러대를 구현하지 못했다.

지금 타이머로 넘기고 있는데 60hz면 16ms로 해도 되나?

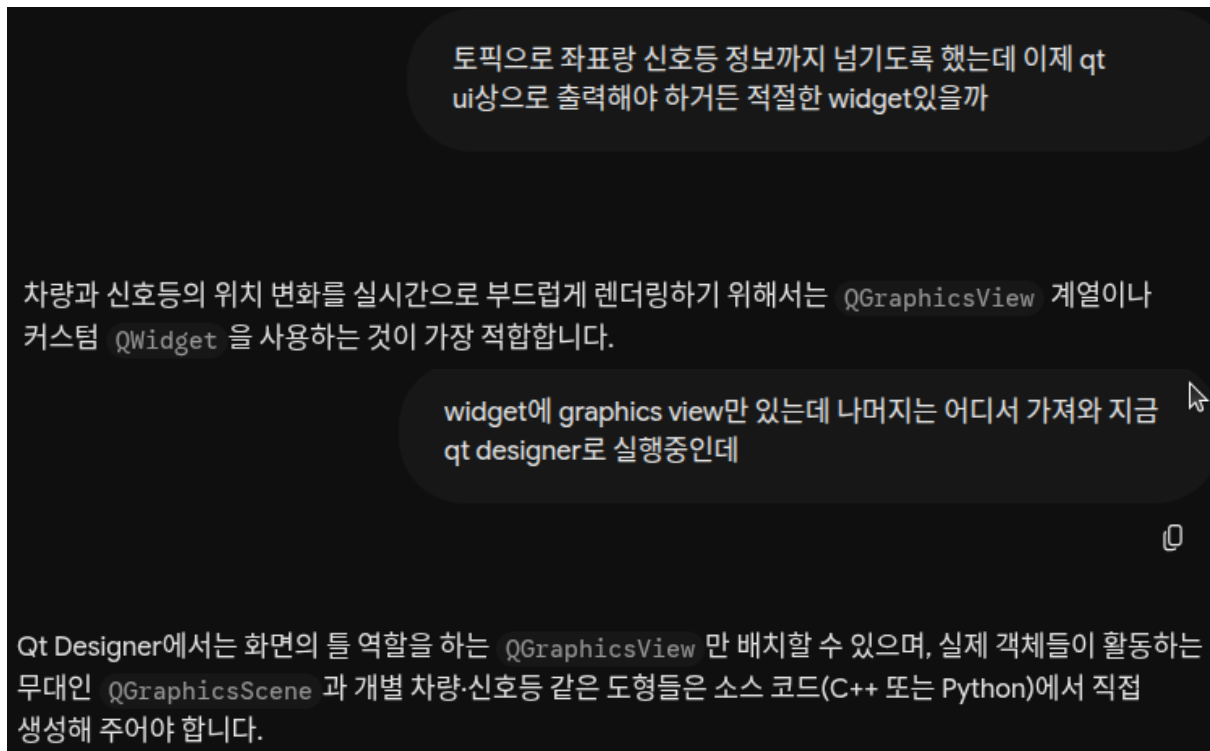
초당 60프레임(60Hz)을 구현하기 위해서는 1000밀리초를 60으로 나눈 약 **16.67ms**가 기준이 되므로, 타이머 주기를 **16ms**로 설정하는 것이 올바릅니다.

차량을 1차원으로만 움직일건데 이러면 앞 뒤 밖에 없으니까 각도 정보는 필요 없지

차량이 1차원 직선 상에서만 이동한다면 동적으로 변하는 각도(Heading) 정보는 필요하지 않습니다.

좌표값을 넘겨줄 때 msg/pose2D를 사용했는데 해당 msg안에 x, y, 세타(각도)값이 담겨져 있다. 근데 초반 계획 상으로도 x축, y축으로만 움직일 계획이었기 때문에 세타값은

publish할 때 보내되, 0을 보내고 sub에서 사용하지 않는 구조로 갔다.



위에서 언급했던 QGrapicsView를 사용하라는 답변을 받았는데 시간이 부족해서 AI가 짜준 UI를 사용하였다.